

ADDRESSING MODES & Instruction set

```
;PROGRAM TO ADD TWO 16-BIT DATA (METHOD-1)
```

```
DATA SEGMENT           ;Assembler directive

    ORG 1104H           ;Assembler directive
    SUM DW 0            ;Assembler directive
    CARRY DB 0          ;Assembler directive

DATA ENDS              ;Assembler directive

CODE SEGMENT           ;Assembler directive

    ASSUME CS:CODE      ;Assembler directive
    ASSUME DS:DATA      ;Assembler directive
    ORG 1000H           ;Assembler directive
```

```
    MOV AX,205AH        ;Load the first data in AX register
    MOV BX,40EDH        ;Load the second data in BX register
    MOV CL,00H          ;Clear the CL register for carry
    ADD AX,BX            ;Add the two data, sum will be in AX
    MOV SUM,AX          ;Store the sum in memory location (1104H)
    JNC AHEAD           ;Check the status of carry flag
    INC CL              ;If carry flag is set,increment CL by one
AHEAD: MOV CARRY,CL      ;Store the carry in memory location (1106H)
    HLT

CODE ENDS              ;Assembler directive
END                   ;Assembler directive
```

Program

A set of instructions written to solve a problem.

Instruction

Directions which a microprocessor follows to execute a task or part of a task.

Computer language

High Level

Low Level

Machine Language

Assembly Language

■ Binary bits

- English Alphabets
- 'Mnemonics'
- **Assembler Mnemonics → Machine Language**

ADDRESSING MODES

Addressing Modes

- Every instruction of a program has to operate on a data. An addressing mode in a microprocessor specifies the way in which the operand of an instruction is chosen — whether it's in a register, memory location, or given directly in the instruction.

1. Register Addressing

2. Immediate Addressing

Group I : Addressing modes for register and immediate data

3. Direct Addressing

4. Register Indirect Addressing

5. Based Addressing

6. Indexed Addressing

7. Based Index Addressing

8. String Addressing

Group II : Addressing modes for memory data

9. Direct I/O port Addressing

10. Indirect I/O port Addressing

Group III : Addressing modes for I/O ports

11. Relative Addressing

Group IV : Relative Addressing mode

12. Implied Addressing

Group V : Implied Addressing mode

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

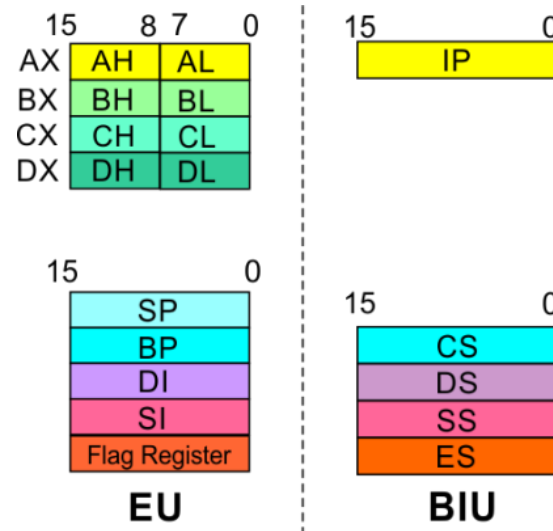
The instruction will specify the name of the register which holds the data to be operated by the instruction.

Example:

MOV CL, DH

The content of 8-bit register DH is moved to another 8-bit register CL

$(CL) \leftarrow (DH)$



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

In immediate addressing mode, the operand (data) is given directly in the instruction itself, instead of referring to memory or a register.

Example:

MOV DL, 08H

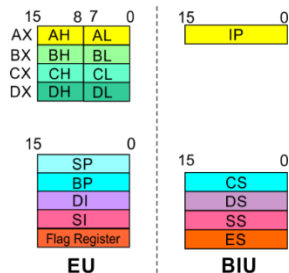
The 8-bit data (08_H) given in the instruction is moved to DL

$(DL) \leftarrow 08_H$

MOV AX, 0A9FH

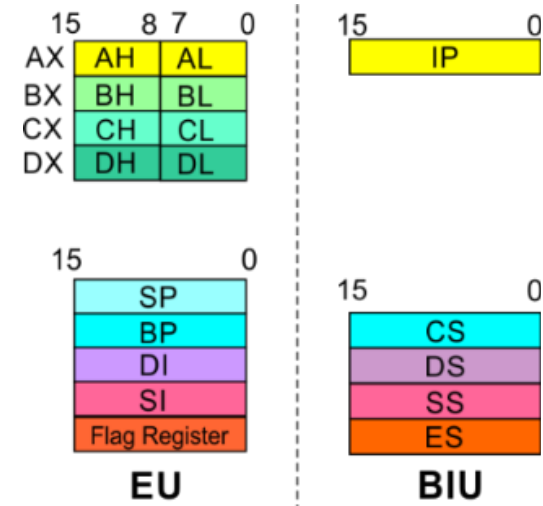
The 16-bit data (0A9F_H) given in the instruction is moved to AX register

$(AX) \leftarrow 0A9F_H$



Addressing Modes : Memory Access

- 20 Address lines $\Rightarrow$ 8086 can address up to $2^{20} = 1\text{M}$ bytes of memory
- However, the largest register is only 16 bits
- Physical Address will have to be calculated
Physical Address : A physical address is the actual location in computer memory (RAM) where data or instructions are stored.
- Memory Address represented in the form
– Seg : Offset (Eg - 89AB:F012)
- Each time the processor wants to access memory, it takes the contents of a segment register, shifts it one hexadecimal place to the left (same as multiplying by 16_{10}), then add the required offset to form the 20-bit address



16 bytes of contiguous memory

89AB : F012 $\rightarrow$ 89AB $\rightarrow$ 89AB0 (Paragraph to byte $\rightarrow 89AB \times 10 = 89AB0$)
 F012 $\rightarrow$ 0F012 (Offset is already in byte unit)
 + -----
 98AC2 (The absolute address)

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

Here, the effective address of the memory location at which the data operand is stored is given in the instruction.

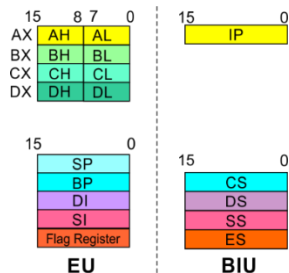
The effective address is just a 16-bit number written directly in the instruction.

Example:

```
MOV BX, [1354H]
MOV BL, [0400H]
```

The square brackets around the 1354_H denotes the contents of the memory location. When executed, this instruction will copy the contents of the memory location into BX register.

In direct addressing mode, the instruction clearly gives the memory location (displacement/ offset) of the operand. This means the CPU can directly find the data's position inside the segment without extra calculation.



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

In Register indirect addressing, name of the register which holds the effective address (EA) will be specified in the instruction.

Registers used to hold EA are any of the following registers:

BX, BP, DI and SI.

Content of the DS register is used for base address calculation.

Example:

MOV CX, [BX]

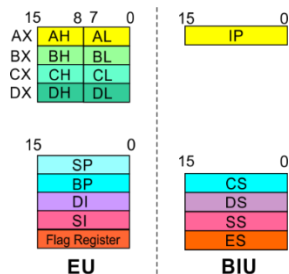
Operations:

EA = (BX)
BA = (DS) × 16₁₀
MA = BA + EA

(CX) ← (MA) or,

(CL) ← (MA)
(CH) ← (MA + 1)

Note : Register/ memory enclosed in brackets refer to content of register/ memory



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

In Based Addressing, BX or BP is used to hold the base value for effective address and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

In case of 8-bit displacement, it is sign extended to 16-bit before adding to the base value.

When BX holds the base value of EA, 20-bit physical address is calculated from BX and DS.

When BP holds the base value of EA, BP and SS is used.

Example:

MOV AX, [BX + 08H]

Operations:

$0008_H \leftarrow 08_H$ (Sign extended)

$EA = (BX) + 0008_H$

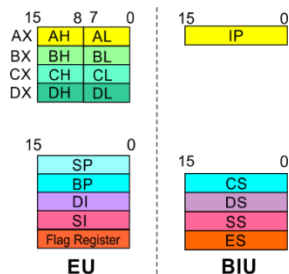
$BA = (DS) \times 16_{10}$

$MA = BA + EA$

$(AX) \leftarrow (MA)$ or,

$(AL) \leftarrow (MA)$

$(AH) \leftarrow (MA + 1)$



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

SI or DI register is used to hold an index value for memory data and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

Displacement is added to the index value in SI or DI register to obtain the EA.

In case of 8-bit displacement, it is sign extended to 16-bit before adding to the base value.

Example:

MOV CX, [SI + 0A2H]

Operations:

$FFA2_H \leftarrow A2_H$ (Sign extended)

$EA = (SI) + FFA2_H$

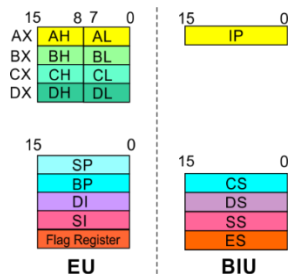
$BA = (DS) \times 16_{10}$

$MA = BA + EA$

$(CX) \leftarrow (MA)$ or,

$(CL) \leftarrow (MA)$

$(CH) \leftarrow (MA + 1)$



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

In Based Index Addressing, the effective address is computed from the sum of a base register (BX or BP), an index register (SI or DI) and a displacement.

Example:

MOV DX, [BX + SI + 0AH]

Operations:

$000A_H \leftarrow 0A_H$ (Sign extended)

$EA = (BX) + (SI) + 000A_H$

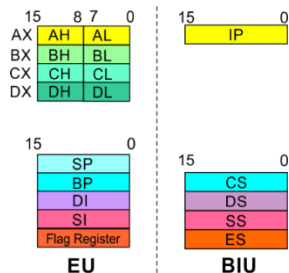
$BA = (DS) \times 16_{10}$

$MA = BA + EA$

$(DX) \leftarrow (MA)$ or,

$(DI) \leftarrow (MA)$

$(DH) \leftarrow (MA + 1)$



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

Note : Effective address of the Extra segment register

String addressing refers to a method where data is processed as a sequence of bytes or words stored in consecutive memory locations

The effective address (EA) of source data is stored in SI register and the EA of destination is stored in DI register.

Segment register for calculating base address of source data is DS and that of the destination data is ES

Example: MOVSB

Operations:

Calculation of source memory location:

$$EA = (SI) \quad BA = (DS) \times 16_{10} \quad MA = BA + EA$$

Calculation of destination memory location:

$$EA_E = (DI) \quad BA_E = (ES) \times 16_{10} \quad MA_E = BA_E + EA_E$$

$$(MAE) \leftarrow (MA)$$

DF=Direction Flag.

If $DF = 1$, then $(SI) \leftarrow (SI) - 1$ and $(DI) \leftarrow (DI) - 1$

If $DF = 0$, then $(SI) \leftarrow (SI) + 1$ and $(DI) \leftarrow (DI) + 1$

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

These addressing modes are used to access data from standard I/O mapped devices or ports.

In direct port addressing mode, an 8-bit port address is directly specified in the instruction.

Example: `IN AL, [09H]`

Operations: $\text{PORT}_{\text{addr}} = 09_{\text{H}}$
 $(\text{AL}) \leftarrow (\text{PORT})$

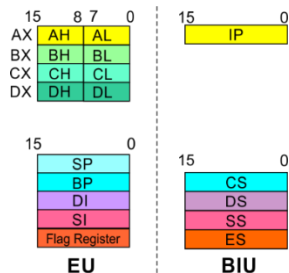
Content of port with address 09_{H} is moved to AL register

In indirect port addressing mode, the port address is specified in the DX register, not directly in the instruction.

Example: `IN AL, DX`

Operations: $\text{PORT}_{\text{addr}} = \text{DX}$
 $(\text{AL}) \leftarrow (\text{PORT})$

The content of the port whose address is in DX is moved to the AL register. This allows access to any 16-bit port using a variable address in DX



I/O Port Addressing in Microprocessors

Microprocessors communicate with external devices (like keyboard, display) using Input/Output (I/O) ports. There are two main methods to access them:

Direct I/O Port Addressing

The port address is directly given in the instruction. The instruction contains the actual port number (8-bit). CPU knows exactly which port to access. The port address is directly given in the instruction.

Example (8085 or 8086):

IN 0x20 ; Read data from I/O port 20h
OUT 0x30 ; Send data to I/O port 30h

0x20 and 0x30 are fixed port addresses.

This is called Direct I/O Addressing.

Indirect I/O Port Addressing

The port address is stored in a register instead of being directly in the instruction. The instruction refers to a register (usually DX in 8086). CPU uses the value in the register as the port address.

Example (8086):

MOV DX, 0x40 ; Store port number in DX
IN AL, DX ; Read from port 40h
OUT DX, AL ; Write to port 40h

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

In this addressing mode, the effective address of a program instruction is specified relative to Instruction Pointer (IP) by an 8-bit signed displacement.

Example: JZ 0AH

Operations:

$000A_H \leftarrow 0A_H$ (sign extend)

If ZF = 1, then

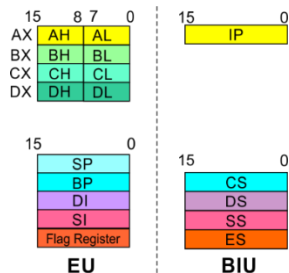
$EA = (IP) + 000A_H$

$BA = (CS) \times 16_{10}$

$MA = BA + EA$

If ZF = 1, then the program control jumps to new address calculated above.

If ZF = 0, then next instruction of the program is executed.



1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

A addressing mode in which the operand is already known by the instruction itself, so no memory address or register is specified.

Example: CLC

This clears the carry flag to zero.

